

Designing and Reporting Simulation Studies

A practical guide for statistics research projects

Irina Gaynanova

Table of contents

1 Why simulation studies	1
2 The four components	2
3 Goal	2
4 Design	3
4.1 Which methods to pick	3
4.2 Designing the data-generating mechanism	3
4.3 Choosing the settings	4
4.4 How many replications?	4
4.5 Which performance measures?	5
5 Implementation	5
5.1 Project layout	5
5.2 Uniform interfaces	6
5.3 Randomness and saved output	8
6 Summary: reporting the results	9
7 Common pitfalls	9
Checklist	10
Appendix: generating structured random objects	10
Example project	11
Further reading	12
License	12

1 Why simulation studies

Simulation is the basic experimental practice of statistical research — the equivalent of basic lab skills in the bench sciences. Its purpose follows from a single observation: to evaluate a method objectively, we must know what the right answer is, and for real data we never do. So we create artificial data where the truth is known, apply the methods to it, and compare the results against that known truth.

This control over the data-generating model is what makes simulation indispensable. It lets us

1. determine how accurate methods are and which perform best, because the truth is known;
2. trace how performance changes as a function of sample size, signal strength, dimension, and other parameters;
3. study how sensitive methods are to their assumptions, by deliberately generating data in which those assumptions are violated.

Simulation plays a role in every kind of statistical research. In **applied** work it justifies the choice of a particular method for the analysis at hand. In **methodological** work it demonstrates numerically that a new method improves on its competitors. In **computational** work it demonstrates improved accuracy or speed. In

theoretical work it validates numerically what the theory predicts. A typical statistics paper devotes an entire section to it, between Method and Data Analysis, and the supplement carries the code that reproduces it.

Comparisons can themselves be the research product: a careful study informing the community about the relative advantages of existing methods is a contribution even when no new method is developed.

1.0.1 Three kinds of data in simulations

Artificial data is generated from a pre-specified statistical model chosen in the context of the methodological question. This is the default meaning of “simulated data” and the focus of this guide.

Real data can also serve, when methods can be evaluated by model-agnostic measures — split the data into a training part for fitting and a test part for evaluation, then compare classification or prediction accuracy on the test set.

Perturbed data is real data artificially disturbed, used to study how robust a method’s results are to perturbation.

1.0.2 Replications

For each fixed design, data is generated many times. Replication accounts for the variability in a method’s performance, and a single draw tells you almost nothing. More replications give more confidence in the comparison at a higher computational cost — §4.4 makes this trade-off precise.

The consequence worth internalizing early: simulation results are *estimates*, computed from a finite number of replications, and carry uncertainty of their own. A study reporting a difference of 0.02 between two methods without quantifying the noise in that 0.02 has not established anything.

2 The four components

Every simulation study has the same four components. Working through them in order, in writing, before any substantial coding, is what separates a study that answers its question from one that merely produces output.

Component	The questions it answers
Goal	What is the context and aim of the comparison?
Design	Which methods? Which data-generating mechanism, with which parameters? Which performance measures? How many replications?
Implementation	Functions for data generation, methods, and metrics; scripts to run the simulation and save results; scripts to turn results into tables and figures; scripts applying the methods to real data
Summary	How are the results communicated in the presentation and the written report?

Readers coming from the published literature will recognize this as a practically-organized version of the **ADEMP** framework of Morris, White, and Crowther (2019) — Aims, Data-generating mechanisms, Estimands, Methods, Performance measures. ADEMP is an excellent checklist for *reporting* a study; the four components above are organized around *doing* one, which is why Implementation appears as a first-class concern here.

3 Goal

State the claim, not the activity. “Compare several methods for linear regression” is an activity. A goal is specific about three things:

- **the method type** — e.g. linear regression of a response y on p covariates $x_1, \dots, x_p$;

- **the specifics** — e.g. situations where p is large, possibly larger than the sample size n , and where not all variables are important;
- **the motivation** — e.g. modern biological datasets are often high-dimensional (1000 genes, 100 samples), and it is biologically plausible that not all genes are needed for prediction.

Stated this way, the goal already constrains the design: if the motivation is $p > n$ with few important variables, then $p > n$ scenarios and sparse coefficient vectors are not optional.

4 Design

4.1 Which methods to pick

Selecting the methods is itself a research task requiring a genuine literature search. Three criteria make a candidate viable:

1. **It falls within the area of interest** defined by the goal.
2. **It has a ready implementation** in R (or whichever language the study uses). A method you must implement from scratch is a project of its own.
3. **It is well described** in textbooks or articles, so that its assumptions and inputs can be understood rather than guessed at.

For the high-dimensional regression example, this yields forward selection, ridge regression, lasso, and principal component regression — each designed to function when $p > n$, each with a mature R implementation, each covered in standard references.

Study each method's implementation closely enough to know its inputs, its outputs, and what it does *not* provide. Discovering mid-study that a method returns loadings but no scores, or requires a tuning parameter you have not planned to choose, is expensive.

Tuning is part of the method. If the lasso penalty is chosen by 5-fold cross-validation, then “lasso + 5-fold CV” is what is being evaluated. Giving one competitor an oracle-tuned parameter while others use cross-validation makes the comparison meaningless.

4.2 Designing the data-generating mechanism

Four questions, in order, take you from the application to a concrete generator.

1. **What is the object we need to create?** Look at the data you actually analyze. For linear regression it is a covariate matrix X and a response vector Y . For a two-group test it is a matrix $X \in \mathbb{R}^{n \times p}$ with labels $Y \in \{0, 1\}^n$. For PCA it is a matrix $X \in \mathbb{R}^{n \times p}$.
2. **What models can create that object?** Look at the assumptions of the methods being compared. Standard linear regression assumes $Y = \beta_0 + X\beta_1 + \varepsilon$ with $\varepsilon_i \sim N(0, \sigma^2)$. PCA methods assume the data is well explained by $k \leq \min(n, p)$ components, $X = UDV^\top + E$ with $V^\top V = I$. Deliberately include models where the assumptions *fail* — correlated errors, a quadratic rather than linear mean, a gamma rather than normal distribution.
3. **What parameters must be selected?** For linear regression: sample size n , number of covariates p , the matrix X , the intercept β_0 , the coefficient vector β_1 , and the noise standard deviation σ . Write out the full list — parameters left implicit become parameters chosen arbitrarily.

4. What do we expect the parameters to do? This question demands the most critical thinking and is the one most often skipped. For linear regression we expect performance to improve as n grows, as σ shrinks, and as the signal β_1 strengthens; and to degrade as p grows. Writing these expectations down before running anything turns the simulation into a genuine test: results that contradict a well-reasoned expectation indicate either an interesting phenomenon or — more often — a bug.

The method's own structure feeds back into the design. Sparse linear regression additionally assumes some elements of β_1 are exactly zero, which implies generating β_1 with exact zeros and considering both large- n /small- p and small- n /large- p settings.

4.3 Choosing the settings

Fix a small number of complete settings — **four to six at most** — rather than an unbounded grid. The discipline is to notice what is common across settings and what differs, and to let the goal decide which differences matter.

A toy example, testing whether a mean equals 98.6 from a sample of size n :

Setting	n	μ	Distribution	What it probes
1	100	98.6	$N(\mu, 1)$	H_0 true, normality holds
2	100	99.6	$N(\mu, 1)$	H_0 false, normality holds
3	100	98.6	gamma, $\alpha\beta = \mu$	H_0 true, normality violated
4	100	99.6	gamma, $\alpha\beta = \mu$	H_0 false, normality violated

These four settings do not cover all possible situations, and they are not meant to. Here n was held fixed at the size of the real data so that two factors that matter more for the goal — whether H_0 holds and whether normality holds — could each be examined. Choosing what *not* to vary is as much a design decision as choosing what to vary.

A study in which the author's preferred method wins in every setting is usually a study whose settings were chosen after seeing the results. Include at least one setting where the favored method is expected to struggle.

4.4 How many replications?

The number of replications is a sample size and deserves justification rather than a default of 1000. Each performance measure is estimated from n_{rep} replications and therefore has a **Monte Carlo standard error** (MCSE); for a mean performance measure,

$$\text{MCSE} = \frac{\widehat{\text{SD}}}{\sqrt{n_{\text{rep}}}}$$

Invert this to plan. If pilot runs suggest $SD \approx 0.5$ and you want to resolve differences of about 0.05, then $n_{\text{rep}} \approx (0.5/0.05)^2 = 100$ gives an MCSE of 0.05 — marginal, since resolving a difference reliably wants an MCSE several times smaller than the difference itself. The calculation takes two minutes and tells you whether the study can answer its question at all.

Report MCSE alongside every headline number. Without it, readers cannot distinguish a real difference from noise — and neither can you.

Computing MCSE in code. Always divide by `sqrt(nrep)` computed from the data rather than a hard-coded constant. Writing `sd(error)/10` works only while `nrep` happens to be 100, and will silently report wrong uncertainties the day you change it.

4.5 Which performance measures?

Performance measures follow from the goal, and the useful discipline is to ask what specifically the method is supposed to recover. Some measures will already be produced by the method wrapper (classification accuracy, a rejection decision); others need dedicated functions (quality of an estimated loading matrix).

Target	Measures
Estimation	Bias, empirical SE, RMSE, ℓ_1 or ℓ_2 error
Prediction	Test-set MSE, misclassification rate
Inference	Type I error control, power, coverage of nominal intervals
Selection	True/false positive rate for selected variables

Two worked examples of the “what should it recover?” question:

For a PCA study — how well is the true number of components k recovered? How well is the true loading matrix V recovered? How closely is X approximated by the recovered components? How well is new test data approximated?

For a testing study — is Type I error controlled at the nominal level? Is the null correctly rejected when the alternative is true? Is classification accuracy high under the alternative? Are the right features selected (for sparse methods)?

Coverage and Type I error deserve emphasis: a method with small bias and badly miscalibrated inference is not a good method, and only a simulation will reveal the miscalibration.

5 Implementation

5.1 Project layout

Organize the repository so that clean, reproducible, well-commented code is separated from exploration:

Functions/	data generation, method wrappers, metric functions
Simulations/	scripts that run the simulation
Outputs/	saved raw results, one file per setting
Figures/	tables and figures produced from saved output
DataAnalysis/	scripts applying the chosen methods to the real data

Testing/ README.md	scratch work and exploration; no prescribed structure what the study does and how to reproduce it
-----------------------	--

Functions/ holds functions **only** — no calls, no plots, no tests. Scripts in Simulations/ bring them in with `source("Functions/Methods.R")` and do the calling. Mixing definitions with the code that exercises them is the fastest way to make a project unreproducible, because sourcing the file then triggers arbitrary side effects.

Version control, repository setup, relative paths, and coding style are treated in the companion **reproducibility** materials; this guide covers only what is specific to simulations.

5.2 Uniform interfaces

The single most important implementation rule: **functions of the same kind must take the same inputs and return the same outputs**. This is what lets the replication loop treat methods interchangeably, and what lets a fourth method be added without touching the loop. On a team, it is also what lets four people write code that composes.

5.2.1 Data generation wrappers

Input: the setting. *Output:* a list containing the generated data and the values of the true parameters needed to evaluate performance later.

The failure mode is subtle and expensive. Two teammates write:

```
# Team member A
generateSim1 <- function(n) {
  # ... generate x with mean mu
  return(list(data = x, mean = mu))
}

# Team member B --- DIFFERENT names, DIFFERENT number of outputs
generateSim2 <- function(n) {
  # ... generate x with mean mu
  return(list(x = x, mu = mu, n = n))
}
```

Every downstream function must now be customized per setting. Agree on the contract first, and both functions return `list(data = x, mean = mu)`.

Where settings share structure, use a master function with a shared sub-function rather than repeating code:

```
generateData <- function(setting) {
  if (setting == 1) {
    n <- 100; mu <- 98.6; distribution <- "normal"
  } else if (setting == 2) {
    n <- 100; mu <- 99.6; distribution <- "normal"
  } else if (setting == 3) {
    n <- 100; mu <- 98.6; distribution <- "gamma"
  } # ... more settings
  x <- drawSample(n, mu, distribution)
```

```

    list(data = x, mean = mu)
  }

drawSample <- function(n, mu, distribution) {
  if (distribution == "normal") {
    rnorm(n, mean = mu)
  } else if (distribution == "gamma") {
    rgamma(n, shape = 1, scale = mu)
  } else {
    stop("Unknown distribution")
  }
}

```

Keeping `n`, `mu`, and `distribution` as explicit arguments of `drawSample` means a setting can be adjusted later without rewriting the generator. A practical team workflow: divide the settings among members and write them as separate functions first, then review each other's code together and refactor the shared structure into sub-functions.

5.2.2 Method wrappers

Input: the output of a data generation function. *Output:* either the performance metrics directly, or the parameters from which performance can be computed.

```

# Classification study --- every method takes and returns the same things
applyKnn <- function(xtrain, ytrain, xtest, ytest) {
  # train the rule on (xtrain, ytrain)
  # evaluate accuracy on (xtest, ytest)
  # record which variables were used, for sparse methods
  list(accuracy = accuracy, ids = ids)
}

# PCA study
applySPCA <- function(X, k) {
  # loadings V (p by k) and scores (n by k), with Scores = XV
  list(V = V, Scores = Scores)
}

# A method that selects the number of components is a different contract
selectK <- function(X) {
  # returns a single integer
}

```

When a procedure wraps a method — a permutation test, say — pass the method in as an argument so the wrapper works for all of them:

```

permuteTest <- function(xtrain, ytrain, xtest, ytest,
                        alpha = 0.05, P = 100, method = applyKnn) {
  out <- method(xtrain, ytrain, xtest, ytest)
  accuracies <- rep(0, P)
  for (i in 1:P) {
    new <- reshuffle(xtrain, ytrain, xtest, ytest)

```

```

    outP <- method(new$xtrain, new$ytrain, new$xtest, new$ytest)
    accuracies[i] <- outP$accuracy
  }
  critical_a <- quantile(accuracies, probs = 1 - alpha)
  list(out = out, critical_a = critical_a,
       decision = out$accuracy > critical_a) # TRUE means reject
}

```

With consistent contracts, generation and application compose without glue code:

```

data    <- generatePCAdataSim1()
outK    <- selectK(data$X)
outSPCA <- applySPCA(data$X, outK)

```

5.2.3 Metric functions

One small function per measure, plus a master `evaluate()` returning all measures for a single replication:

```

MSE <- function(yhat, Y) mean((yhat - Y)^2)

L1EstimationError <- function(hatbeta, truebeta) sum(abs(truebeta - hatbeta))

evaluate <- function(hatbeta0, hatbeta, truebeta, Y_test, X_test) {
  list("MSE Test" = MSE(yhat(hatbeta0, hatbeta, X_test), Y_test),
       "L1 Norm"  = L1EstimationError(hatbeta, truebeta))
}

```

5.3 Randomness and saved output

Three practices specific to simulations cover most of the reproducibility risk. (Version control and project setup belong to the reproducibility materials; what follows is about randomness and results.)

Set the seed in the script, once. Record it in the file rather than typing it at the console, so the script alone reproduces the run.

Use a parallel-safe random number generator. This is the pitfall that silently invalidates results. Ordinary `set.seed()` does *not* make a parallel `foreach` loop reproducible, because workers receive streams in a non-deterministic order. The `doRNG` package solves this: `%dorng%` gives each replication its own reproducible stream, so results are identical across runs *and* across different numbers of workers.

```

library(doParallel); library(doRNG)
cl <- makeCluster(detectCores()); registerDoParallel(cl)
set.seed(12345)

results <- foreach(i = 1:nrep, .packages = c("glmnet", "mnormt", "pls")) %dorng% {
  # ... one replication; the returned object is all that is kept
}
stopCluster(cl)
save(results, file = paste0("Simulations/Outputs/n", n_train, "p", p, ".Rdata"))

```

Note `.packages`: parallel workers start in fresh R sessions and do not inherit your loaded libraries. Every package used inside the loop must be listed, and a missing entry produces “could not find function” errors that appear only in parallel, often after a long run.

Save raw per-replication output, not summaries. Simulations are expensive and plots are not, so the replication script saves raw results and stops; a separate script loads them and produces figures and tables. This makes new questions (“what does the *distribution* look like?”, “is the mean driven by three outliers?”) answerable without recomputing anything, and you will redraw a figure twenty times before the report is done. Name output files after the setting they came from — `n50p10.Rdata` — so results are never ambiguous.

6 Summary: reporting the results

Show distributions, not just means. Boxplots or violin plots across replications reveal skewness, outliers, and occasional catastrophic failures that a table of means hides entirely. A method that is best on average and fails badly in 5% of replications is important to identify.

```
pred_data.long |>
  ggplot(aes(x = method, y = error, fill = method)) +
  geom_boxplot() +
  facet_grid(~ metric)
```

Generate tables and figures from scripts, never by hand. Point-and-click export breaks reproducibility, and hand-typed numbers introduce errors that nobody catches. Write the figure to a file and build tables programmatically.

Use a consistent layout across settings. Same method order, same colors, same axis scales wherever comparison is intended. Readers compare panels visually, and inconsistent scales make honest comparison impossible.

Report failures. Non-convergence, errors, and dropped replications are results. State how many replications failed for each method and how they were handled; quietly discarding them biases every number in the table.

Give enough detail to reproduce. State n_{rep} , the seed, the software versions, and the parameter values for every setting. A reader should be able to reconstruct the design from the report alone.

7 Common pitfalls

Pitfall	Why it matters
Settings chosen after seeing results	Turns a study into a demonstration; fix the design in advance
Only favorable settings	Reviewers will find the missing case; better that you find it first
No expectations written down	Removes the check that would have caught the bug
Inconsistent wrapper input/output	Forces per-setting customization of everything downstream
Functions mixed with scripts and plots	Sourcing triggers side effects; nothing is reusable
No Monte Carlo SE	Differences cannot be distinguished from noise
Unfair tuning	Oracle-tuning one method invalidates the comparison
<code>set.seed()</code> with parallel loops	Results are not reproducible; use <code>doRNG</code>
Missing <code>.packages</code> in <code>foreach</code>	Fails only in parallel, often after a long run

Pitfall	Why it matters
Saving summaries only	Any new question requires re-running everything
Discarding failed replications silently	Biases results toward methods that fail on hard cases
Figures saved by point-and-click	Cannot be regenerated; breaks reproducibility

Checklist

Before considering a simulation study finished:

- ☐ Goal stated with method type, specifics, and motivation
- ☐ Methods selected by literature search; each has an R implementation
- ☐ Tuning procedure specified and fair across methods
- ☐ Data-generating mechanism derived from the four design questions
- ☐ Four to six settings fixed, with the reason for each
- ☐ At least one setting where the favored method should struggle
- ☐ Expectations for each parameter written down before running
- ☐ Performance measures matched to what methods should recover
- ☐ n_{rep} justified by a Monte Carlo SE calculation
- ☐ All wrappers of the same kind share input/output contracts
- ☐ Functions/ contains function definitions only
- ☐ Seed set in-script; doRNG used if running in parallel
- ☐ Raw per-replication output saved, named by setting
- ☐ Figures and tables generated by script, not by hand
- ☐ Figures show distributions; tables report MCSE
- ☐ Failed replications counted and reported
- ☐ README explains how to reproduce every figure and table

Appendix: generating structured random objects

Recipes that recur when building data-generating mechanisms.

Random orthogonal matrix ($p \times k$ with $V^T V = I$):

```
k <- 3; p <- 40
N <- matrix(rnorm(k * p), p, k)
V <- svd(N)$u[, 1:k]
round(crossprod(V), 2) # identity
```

The randortho function in the pracma package does the same thing.

Sparse unit-norm vector (s non-zeros in random positions):

```
p <- 10; s <- 3
v <- c(runif(s), rep(0, p - s))
v <- sample(v) # permute positions
v <- v / sqrt(sum(v^2)) # standardize to norm 1
```

Sparse orthogonal loading matrix, built column by column so orthogonality holds:

```

k <- 3; p <- 10; s <- 3
V <- matrix(0, p, k)
v <- sample(c(runif(s), rep(0, p - s)))
V[, 1] <- v / sqrt(sum(v^2))
for (j in 2:k) {
  v <- sample(c(runif(s), rep(0, p - s)))
  v <- v - tcrossprod(V[, 1:(j - 1)]) %*% v # project out previous columns
  V[, j] <- v / sqrt(sum(v^2))
}

```

Equi-correlation covariance matrix — more realistic than the identity:

```

p <- 5; rho <- 0.7
Sigma <- matrix(rho, p, p) + diag(1 - rho, p)

```

Data-driven covariance — estimate from real data and regularize slightly:

```

Sigma <- cor(data)
Sigma <- 0.9 * Sigma + 0.1 * diag(ncol(data))

```

Multivariate normal sample with given mean and covariance:

```

library(MASS)
X <- mvrnorm(n, mu, Sigma)

```

Example project

A companion repository, [simulation-project-template](#), compares ridge regression, lasso, and principal component regression under a linear model with correlated covariates. It is deliberately small enough to read in one sitting, and demonstrates the separation of concerns, the uniform wrapper interfaces, parallel replication with doRNG, saved raw output, and a separate visualization script:

DataGeneration.R	generate X and Y; generate a sparse beta
Methods.R	applyRidge, applyLasso, applyPCR
Metrics.R	MSE, L1 error, master evaluate()
n50_p10_parallel_replications.R	the experiment; saves n50p10.Rdata
Visualize_results.R	loads results, makes boxplots and a table
n50p10.Rdata	saved raw per-replication output
Boxplots_comparison.pdf	the resulting figure

Because the project is small, the files sit in one flat directory rather than in the Functions/ and Simulations/ folders of §5.1; the separation of concerns is the same, and the folder layout becomes worthwhile as soon as a study has several settings and multiple output files.

It is set up as a GitHub **template repository**: click *Use this template* to create your own copy, then clone it and open the .Rproj file so that relative paths resolve correctly. Replace the data generation functions with your settings, the method wrappers with your methods, and the metric functions with your performance measures. The replication and analysis scripts will need almost no modification, which is the point of the uniform interfaces.

Further reading

Morris TP, White IR, Crowther MJ (2019). Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11), 2074–2102. <https://doi.org/10.1002/sim.8086> — the standard modern reference; source of the ADEMP framework and detailed guidance on performance measures and Monte Carlo error.

Burton A, Altman DG, Royston P, Holder RL (2006). The design of simulation studies in medical statistics. *Statistics in Medicine*, 25(24), 4279–4292. <https://doi.org/10.1002/sim.2673> — an earlier and still valuable treatment of design choices.

Boulesteix AL, Groenwold RHH, Abrahamowicz M, et al. (2020). Introduction to statistical simulations in health research. *BMJ Open*, 10(12), e039921. <https://doi.org/10.1136/bmjopen-2020-039921> — accessible introduction aimed at applied researchers, with a useful treatment of performance metrics.

Sigal MJ, Chalmers RP (2016). Play it again: teaching statistics with Monte Carlo simulation. *Journal of Statistics Education*, 24(3), 136–156. <https://doi.org/10.1080/10691898.2016.1246953> — simulation from a teaching perspective, with the workflow laid out step by step.

Chalmers RP, Adkins MC (2020). Writing effective and reliable Monte Carlo simulations with the SimDesign package. *The Quantitative Methods for Psychology*, 16(4), 248–280. <https://doi.org/10.20982/tqmp.16.4.p248> — a framework-based alternative to hand-rolled simulation code in R.

Wilson G, Bryan J, Cranston K, Kitzes J, Nederbragt L, Teal TK (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510> — project organization, file naming, and data management habits that apply directly.

License

“Designing and Reporting Simulation Studies” © 2026 by Irina Gaynanova is licensed under the [Creative Commons Attribution 4.0 International License](#) (CC BY 4.0). You are free to share and adapt this guide for any purpose, including commercially, provided you give appropriate credit, link to the license, and indicate if changes were made. Suggested attribution: “Adapted from Designing and Reporting Simulation Studies by Irina Gaynanova, CC BY 4.0.”